Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251211**
Nama Lengkap	**Jendri Mansamnaf Japen**
Minggu ke / Materi	**09 / Pengolahan String dan Regular Expression**

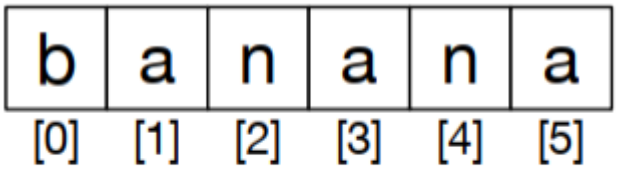
PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

8.3.2 Pengaksesan String dan Manipulasi String

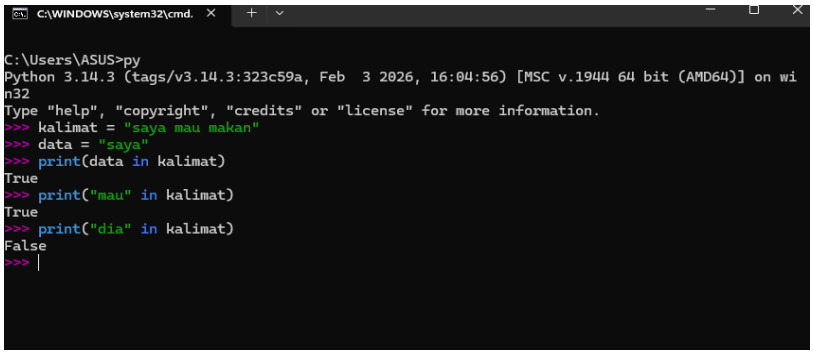
Jadi string pertama kali dibuat dengan deklarasi variabel dan langsung diisi dengan data, string dapat diakses sebagai satu kesatuan dengan menyebut nama variabelnya, atau per huruf dengan menyebutkan indeksnya. Indeks pada string dimulai dari 0 seperti layaknya pada list. Indeks string haruslah berupa bilangan bulat, bukan pecahan. Pada memory komputer, string disimpan secara urut menggunakan list yang berisi huruf-huruf dengan indeks yang dimulai dari nol. Gambar 8.1 menunjukkan string berisi "banana" yang disimpan pada memory komputer.



Gambar 8.1: Bentuk string di dalam memory

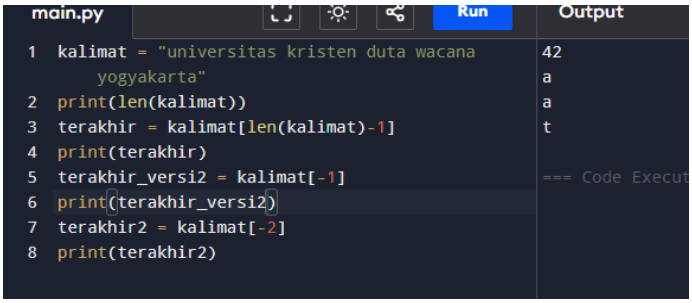
8.3.3 Operator dan Metode String OPERATOR in

Pada String kita dapat memeriksa apakah suatu kalimat merupakan substring dari suatu kalimat lain dengan menggunakan operator in. Hasil darioperator ini adalah True / False.



FUNGSI len

Cara untuk mengetahui berapa panjang (berapa jumlah karakter) dari sebuah string adalah dengan menggunakan operator len() Untuk menampilkan huruf terakhir dari sebuah string kita harus menggunakan indeks string yang ke- len(-1), sebab indeks dimulai dari 0. Contoh program Python adalah:



TRAVERSING STRING

Ada 2 cara menampilkan string huruf per huruf:

Menggunakan indeks (while) => mengakses tiap huruf dengan posisi (kalimat[i]) menggunakan perulangan while.

Tanpa indeks (for) => langsung mengambil tiap huruf dari string menggunakan perulangan for.

Keduanya sama-sama menampilkan huruf satu per satu, tetapi for lebih sederhana dan praktis, sedangkan while memberi kontrol lewat indeks.

STRING SLICE

String slice adalah menampilkan substring pada sebuah string dengan menggunakan indeks dari awal tertentu sampai akhir-1 tertentu. Sintaksnya menggunakan [awal:akhir]. Bagian awal atau akhir boleh dikosongkan. Bagian awal dimulai dari 0.

Nama Method	Kegunaan	Penggunaan
capitalize()	untuk mengubah string menjadi huruf besar	string.capitalize()
count()	menghitung jumlah substring yang muncul dari sebuah string	string.count()
endswith()	mengetahui apakah suatu string diakhiri dengan string yang dinputkan	string.endswith()
startswith()	mengetahui apakah suatu string diawali dengan string yang dinputkan	string.startswith()
find()	mengembalikan indeks pertama string jika ditemukan string yang dicari	string.find()
islower() dan isupper()	mengembalikan True jika string adalah huruf kecil / huruf besar	string.islower() dan string.isupper()
isdigit()	mengembalikan True jika string adalah digit (angka)	string.isdigit()
strip()	menghapus semua whitespace yang ada di depan dan di akhir string	string.strip()
split()	memecah string menjadi token-token berdasarkan pemisah, misalnya berdasarkan spasi	string.split()

Dan masih banyak lainnya. Ingat semua fungsi/method di atas yang mengembalikan string, mengembalikan string baru, tidak mengubah yang aslinya, karena string bersifat

immutable. Daftar lengkap dapat dilihat di https://docs.python.org/library/stdtypes.html#string-methods.

Operator * dan + pada String

Pada Python operator + dan * memiliki kemampuan khusus. Operator + yang biasanya digunakan untuk menjumlahkan bilangan bisa digunakan untuk menggabungkan dua buah string. Sedangkan operator * yang bisa digunakan untuk mengkalikan bilangan bisa digunakan untuk menampilkan string sejumlah perkaliannya. Perhatikan kode berikut:

```
kata1 = "saya"
kata2 = "makan"
kata3 = kata1 + " " + kata2
print(kata3)     #hasil adalah penggabungan: saya makan
kata4 = "ulang"
print(kata4 * 4)     #hasil adalah ulangulangulangulang
kata4 = "ulang "
print(kata4 * 2)     #hasil adalah ulang ulang
```

8.3.4 Parsing String

Parsing string adalah cara menelusuri string bagian demi bagian untuk mendapatkan / menemukan / mengubah bagian string yang diinginkan. Perhatikan contoh berikut: "Saudara-saudara, pada tanggal 17-08-1945 Indonesia merdeka". Dilihat dari string di atas, diingikan untuk mengambil tanggal bulan tahun dari kalimat di atas dan disusun ulang menjadi format 08/17/1945. Kita dapat menggunakan cara parsing string misalnya: • Spit string berdasarkan spasi sehingga menjadi token: "Saudara-saudara", "pada", "tanggal", "17-08-1945", "Indonesia", dan "merdeka". • Dari sini, lanjutkan dengan mencari token yang diawali dengan angka, kemudian lanjutkan dengan split angka tersebut dengan pemisah '-'. • Kemudian susun ulang token-token dari langkah sebelumnya sehingga berbentuk seperti yang diinginkan. Berikut adalah contoh penyelesaiannya:

```
kalimat = "Saudara-saudara, pada tanggal 17-08-1945 Indonesia merdeka"

hasil = kalimat.split(" ")

for kal in hasil:
    if kal[0].isdigit():
        hasil2 = kal.split("-")
        print(hasil2[1]+"/"+hasil2[0]+"/"+hasil2[2])
```

Hasil dari keluaran program adalah **08/17/1945**

8.3.5 Pengantar Regex

Pada bab String, kita sudah sedikit memperlajari mengenai teknik-teknik pengaksesan string, manipulasi string, dan berbagai kasus-kasus pengolahan string lainnya, termasuk string yang terdapat pada file. Dari pengalaman tersebut dapat dilihat bahwa kita cukup kesulitan untuk melakukan pengolahan string dengan teknik biasa / standar. Terdapat teknik pengolahan string yang lebih mudah dan cepat dengan menggunakan bantuan regular expression. Regular expression adalah ekspresi pola yang berbentuk kumpulan karakter yang digunakan untuk menemukan pola (pattern) yang sama dengan pola regex di dalam string lain yang ingin dicari. Regex membantu kita dalam pencarian string dengan pola tertentu,

mengganti string dengan pola tertentu, dan menghapus string dengan pola tertentu. Intinya regex membantu dalam parsing string yang selama ini biasanya hanya menggunakan perintah split() dan find() saja. Regex sangat powerful dalam searching dan extracting pola namun memiliki pola yang cukup rumit. Tidak semua bahasa pemrograman mendukung regular expression library. Python merupakan salah satu bahasa yang mendukung library regex dengan cara import re. Salah satu fungsi yang paling mudah digunakan dari library re adalah search(). Dengan menggunakn file mbox-short.txt, kita akan mencoba menampilkan semua string pada file tersebut yang mengandung pola "From: ".

```
import re
handle=open('mbox-short.txt')
count = 0
for line in handle:
line=line.rstrip()
if re.search('From:', line):
count += 1
print(line)
print("Count: ",count)
```

Dari kode di atas kita dapat melihat bahwa re.search bisa saja diganti dengan menggunakan perintah find() pada string biasa. Pola pada contoh di atas belum menggunakan kemampuan regex yang seutuhnya.

From: stephen.marquard@uct.ac.za

From: louis@media.berkeley.edu

From: zqian@umich.edu

From: rjlowe@iupui.edu

From: zqian@umich.edu

From: rjlowe@iupui.edu

From: cwen@iupui.edu

From: cwen@iupui.edu

From: gsilver@umich.edu

From: gsilver@umich.edu

From: zqian@umich.edu

From: gsilver@umich.edu

From: wagnermr@iupui.edu

From: zqian@umich.edu

From: antranig@caret.cam.ac.uk

From: gopal.ramasammycook@gmail.com

From: david.horwitz@uct.ac.za

From: david.horwitz@uct.ac.za

From: david.horwitz@uct.ac.za

From: david.horwitz@uct.ac.za

From: stephen.marquard@uct.ac.za

From: louis@media.berkeley.edu

From: louis@media.berkeley.edu

From: ray@media.berkeley.edu

From: cwen@iupui.edu

From: cwen@iupui.edu

From: cwen@iupui.edu

Jumlah: 27

Jika diinginkan mencari baris yang diawali dengan pola "From", maka kita harus mengubah parameter fungsi search pada re.search menjadi re.search("From") ^ .

```
import re
handle=open('mbox-short.txt')
count = 0
for line in handle:
line=line.rstrip()
if re.search('^From:', line):
count += 1
print(line)
print("Count: ",count)
```

Kode di atas dapat digantikan juga dengan fungsi string.startswith("From :").

8.3.6 Meta Character, Escaped Character, Set of Character, dan Fungsi Regex pada Libary Python

Sebelum menggunakan fungsi regex perlu diketahui terlebih dahulu meta character / special character dan kegunaannya pada pola regex seperti pada tabel 8.1

Tabel 8.1: Special Character pada Python

Karakter	Kegunaan	Contoh	Arti Contoh
[]	Kumpulan karakter	"[a-zA-Z]"	1 karakter antara a-z kecil atau A-Z besar
\{}	Karakter dengan arti khusus dan escaped character	\{}d	Angka / digit
.	Karakter apapun kecuali newline	say.n.	Tidak bisa diganti dengan karakter apapun, misal "sayang" akan valid
^	Diawali dengan	^From	Diawali dengan From
$	Dakhiri dengan	this$	Diakhiri dengan kata this
*	0 s/d tak terhingga karakter	\{}d*	ada digit minimal 0 maksimal tak terhingga
?	ada atau tidak (opsional)	\{}d?	Boleh ada atau tidak ada digit sebanyak
+	1 s/d tak terhingga karakter	\{}d+	Minimal 1 s/d tak terhingga karakter
{}	Tepat sebanyak yang ada para {}	\{}d{2}	Ada tepat 2 digit
()	Pengelompokan karakter / pola	(sayalkamu)	saya atau kamu sebagai satu kesatuan
|	atau	\{}d|\{}s	1 digit atau 1 spasi

Tabel 8.2: Escaped Character pada Regex

Special Characters	Kegunaan	Contoh
\b	Digunakan untuk mengetahui apakah suatu pola berada di awal kata atau akhir kata	"R\bin" "Rain\b"
\d	Digunakan untuk mengetahui apakah karakter adalah sebuah digit (0 s/d 9)	\d
\D	Digunakan untuk mengetahui apakah karakter yang bukan digit	\D
\s	Digunakan untuk mengetahui apakah karakter adalah whitespace (spasi, tab, enter)	\s
\S	Digunakan untuk mengetahui apakah karakter adalah BUKAN whitespace (spasi, tab, enter)	\S
\w	Digunakan untuk mengetahui apakah karakter adalah word (a-z, A-Z, 0-9, dan _)	\w
\W	Digunakan untuk mengetahui apakah karakter adalah BUKAN word (a-z, A-Z, 0-9, dan _)	\W
\A	Digunakan untuk mengetahui apakah karakter adalah berada di bagian depan dari kalimat	"\AThe"
\Z	Digunakan untuk mengetahui apakah karakter adalah berada di bagian akhir dari kalimat	"End\Z"

Pada Python terdapat beberapa special character (espcaped characters) seperti pada tabel 8.2 Pada Python terdapat beberapa penggunaan himpunan character dengan menggunakan simbol [], pada tabel 8.3

Tabel 8.3: Himpunan Karakter pada Regex

[abc]	Mencari pola 1 huruf a, atau b, atau c
[a-c]	Mencari pola 1 huruf a s/d c
[^bmx]	Mencari pola 1 huruf yang bukan b,m, atau x
[012]	Mencari pola 1 huruf 0, atau 1, atau 2
[0-3]	Mencari pola 1 huruf 0 s/d 3
[0-2][1-3]	Mencari pola 2 huruf: 01, 02, 03, 11, 12, 13, 21, 22, 23
[a-zA-Z]	Mencari pola 1 huruf a-Z

Pada Python terdapat 4 buah fungsi yang bisa dipakai untuk menggunakan Regex seperti pada tabel 8.4

Tabel 8.4: Fungsi Regex pada Python

Nama Fungsi	Kegunaan
findall	mengembalikan semua string yang sesuai pola (matches)
search	mengembalikan string yang sesuai pola (match)
split	memecah string sesuai pola
sub	mengganti string sesuai dengan pola yang cocok

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Latihan 8.1

```
[4]: #Latihan 8.1
def cek_anagram(kata1, kata2):
    kata1 = kata1.replace(" ", "").lower()
    kata2 = kata2.replace(" ", "").lower()

    if sorted(kata1) == sorted(kata2):
        return "Anagram"
    else:
        return "Bukan anagram"

kata1 = input("Masukan kata pertama: ")
kata2 = input("Masukan kata kedua: ")

hasil = cek_anagram(kata1, kata2)
print("Hasil: ", hasil)
```

```
Masukan kata pertama:  mata
Masukan kata kedua:  maat
Hasil:  Anagram
```

Program ini mengecek apakah dua kata adalah anagram dengan cara mengubah huruf menjadi kecil, menghapus spasi, lalu mengurutkan hurufnya. Jika hasil urutannya sama, maka anagram, jika tidak maka bukan.

Latihan 8.2

```
[9]: #latihan 8.2
def hitung_kata(kalimat, kata_dicari):
    kalimat = kalimat.lower()
    kata_dicari = kata_dicari.lower()

    kalimat = kalimat.replace(".", "").replace(",", "")
    daftar_kata = kalimat.split()

    jumlah =daftar_kata.count(kata_dicari)
    return jumlah

kalimat = input("Masukan kalimat: ")
kata = input("Masukan kata yang dicari: ")

hasil = hitung_kata(kalimat, kata)
print(f"{kata} ada {hasil} buah")
```

```
Masukan kalimat:  Saya mau makan. Makan itu wajib. Mau siang atau malam saya wajib makan.
Masukan kata yang dicari:  makan
makan ada 3 buah
```

Program ini menghitung berapa kali suatu kata muncul dalam kalimat dengan cara mengubah semua huruf menjadi kecil, menghapus tanda baca, memisahkan kalimat menjadi kata-kata, lalu menghitung jumlah kemunculan kata yang dicari.

Latihan 8.3

```
[11]: #latihan 8.3
def hapus_spasi_berlebihan(teks):
    hasil = " ".join(teks.split())
    return hasil

kalimat = input("Masukan kalimat: ")
hasil = hapus_spasi_berlebihan(kalimat)
print("Hasil:", hasil)
```

```
Masukan kalimat:  saya  tidak  suka  makan  ikan .
Hasil: saya tidak suka makan ikan .
```

Program ini menghapus spasi berlebih pada kalimat dengan memisahkan kata-kata lalu menggabungkannya kembali menggunakan satu spasi sehingga hasilnya rapi.

Latihan 8.4

```
[12]: #latihan 8.4
def cari_kata(kalimat):
    kalimat = kalimat.replace(".", "").replace(",", "")
    daftar_kata = kalimat.split()
    terpendek = min(daftar_kata, key=len)
    terpanjang = max(daftar_kata, key=max)
    return terpendek, terpanjang

kalimat = input("Masukan kalimat: ")
pendek, panjang = cari_kata(kalimat)

print("Terpendek:", pendek)
print("Terpanjang:", panjang)
```

```
Masukan kalimat:  red snakes and a black frog in the pool
Terpendek: a
Terpanjang: the
```

Program ini menentukan kata terpendek dan terpanjang dengan memisahkan kalimat menjadi kata-kata, lalu membandingkan panjang setiap kata untuk mendapatkan yang paling pendek dan paling panjang.

Latihan 8.5

```
[15]: #Latihan 8.5
import re
from datetime import datetime

def cari_tanggal(teks):
    pola = r'\d{4}-\d{2}-\d{2}'
    daftar_tanggal = re.findall(pola, teks)
    sekarang = datetime.now()

    for tgl in daftar_tanggal:
        tanggal = datetime.strptime(tgl, "%Y-%m-%d")
        selisih = (sekarang - tanggal).days
        print(tanggal, "selisih", selisih, "hari")
teks = input("Masukan teks: ")
cari_tanggal(teks)
```

```
Masukan teks:  Pada tanggal 1945-08-17 Indonesia merdeka. Indonesia memiliki beberapa pahlawan nasional, seperti Pangeran Diponegoro (TL: 1785-11-1
1), Pattimura (TL: 1783-06-08) dan Ki Hajar Dewantara (1889-05-02).
1945-08-17 00:00:00 selisih 29464 hari
1785-11-11 00:00:00 selisih 87816 hari
1783-06-08 00:00:00 selisih 88703 hari
1889-05-02 00:00:00 selisih 50024 hari
```

Program ini mencari semua tanggal berformat YYYY-MM-DD menggunakan regex, lalu mengubahnya ke tipe datetime dan menghitung selisih hari dengan tanggal sekarang untuk ditampilkan.

Latihan 8.6

```
[18]: #latihan 8.6
import re
import random
import string

def generate_password(panjang=8):
    karakter = string.ascii_letters + string.digits
    return ''.join(random.choice(karakter) for _ in range(panjang))

def cari_email(teks):
    pola = r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'
    daftar_email = re.findall(pola, teks)

    for email in daftar_email:
        username = email.split("@")[0]
        password = generate_password()
        print(f"{email} username: {username} , password: {password}")

teks = input("Masukan teks: ")
cari_email(teks)
```

```
Masukan teks:  anton@mail.com
anton@mail.com username: anton , password: T5axxWKp
```

Program ini mencari semua email dalam teks menggunakan regex, mengambil username dari bagian sebelum @, lalu menghasilkan password acak 8 karakter yang terdiri dari huruf dan angka untuk setiap email.

Link github: https://github.com/japenjendri-design/Praktikkum-Algoritma-dan-Pemrograman.git